# SECURE CODE AUDIT REPORT

*Task 3 — Secure Coding Review*

CodeAlpha Cybersecurity Internship Program

*Date: 26 May 2026  Severity: OWASP Risk Rating Methodology*

| 3 | 1 | 5 | Python / Flask |
|---|---|---|---|
| Critical findings | High findings | OWASP categories | Language audited |

# 1. Executive Summary

This report presents the findings of a manual secure code review conducted on a Python/Flask application as part of the CodeAlpha Cybersecurity Internship program. The audit identified four security vulnerabilities across three code modules, covering five OWASP Top 10 categories.

All findings were verified through proof-of-concept analysis. Remediation code is provided for each finding. The most critical issues — SQL Injection and Broken Authentication — must be addressed before any production deployment.

| ID | Title | Severity | OWASP Category | Status |
|---|---|---|---|---|
| 01 | SQL Injection | **CRITICAL** | A01 — Injection | **Remediated** |
| 02a | Plaintext Password Storage | **CRITICAL** | A02 — Broken Auth | **Remediated** |
| 02b | Sensitive Data in Logs | **HIGH** | A03 — Data Exposure | **Remediated** |
| 03a | Reflected XSS | **HIGH** | A03 — XSS | **Remediated** |
| 03b | Debug Mode in Production | **CRITICAL** | A05 — Misconfiguration | **Remediated** |

# 2. Detailed Findings

### Finding 01 — SQL Injection

| | |
|---|---|
| **Severity** | **CRITICAL** |
| **Category** | OWASP A01 — Injection |
| **Location** | function get_user(), line 6 — database.py |
| **Description** | User input is concatenated directly into an SQL query string without Pipeline sanitization. An attacker can inject arbitrary SQL to read, modify, or delete any data in the database. |
| **Proof of concept** | Input ' OR '1'='1 returns all rows from the users table. |
| **Remediation** | Use parameterized queries. Replace string concatenation with cursor.execute(query, (param,)) so user input is never interpreted as SQL. |

**Vulnerable code:**

```
query = "SELECT * FROM users WHERE username = '" + username + "'"
cursor.execute(query)
```

**Fixed code:**

```
query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,))
```

## Finding 02 — Broken Authentication + Sensitive Data Exposure

| | |
|---|---|
| **Severity** | **CRITICAL** |
| **Category** | OWASP A02 + A03 |
| **Location** | function login(), lines 11-13 — auth.py |
| **Description** | Passwords are stored and compared in cleartext. Login credentials are also written to application logs, exposing them to anyone with log access. |
| **Proof of concept** | Credentials stored as plaintext. Log file exposes passwords on every login attempt. |
| **Remediation** | Hash passwords with bcrypt before storing. Use bcrypt.checkpw() for comparison. Never log credentials — log only username and success/failure status. |

**Vulnerable code:**

```
if stored_password == password:  # plaintext comparison
    return True
print(f"Login: username={username}, password={password}")  # NEVER log passwords
```

**Fixed code:**

```
if bcrypt.checkpw(password.encode(), stored_hash):
    logging.info(f"Login success: username={username}")  # no password logged
    return True
```

## Finding 03 — Reflected XSS + Security Misconfiguration

| | |
|---|---|
| **Severity** | **HIGH** |
| **Category** | OWASP A03 + A05 |
| **Location** | route /search — app.py |
| **Description** | User input from URL is rendered directly into HTML without escaping, enabling reflected XSS. debug=True also exposes an interactive Python console on error pages (Remote Code Execution risk). |
| **Proof of concept** | GET /search?q=<script>alert('XSS')</script> executes JavaScript in victim browser. |
| **Remediation** | Escape all user input with Flask's escape() or Jinja2 auto-escaping. Always set debug=False in production — use environment variables instead. |

**Vulnerable code:**

```
return f"<h2>Results for: {query}</h2>"  # unescaped input - XSS
app.run(debug=True)  # exposes Python console in production
```

**Fixed code:**

```
safe_query = escape(query)  # converts < > to &lt; &gt;
return f"<h2>Results for: {safe_query}</h2>"
app.run(debug=False)
```

# 3. Recommendations

### Immediate actions (before any deployment)

- Replace all string-concatenated SQL queries with parameterized statements.
- Migrate all stored passwords to bcrypt hashes — invalidate existing plaintext passwords.
- Audit all log statements and remove any that record credentials or sensitive tokens.
- Disable debug mode — use environment variable: debug=os.getenv('DEBUG', False).
- Enable Jinja2 auto-escaping for all Flask templates (enabled by default for .html files).

### Secure coding best practices

- Follow the OWASP Secure Coding Practices Quick Reference Guide for all new code.
- Integrate Bandit (static analysis for Python) into the CI/CD pipeline.
- Conduct security-focused code reviews using a checklist before merging pull requests.
- Store all secrets and configuration in environment variables, never in source code.
- Apply the principle of least privilege: database users should have minimal permissions.

# 4. Tools & References

Static analysis: Bandit for Python — pip install bandit && bandit -r ./project

OWASP Top 10 2021: https://owasp.org/www-project-top-10/

OWASP Secure Coding Practices: https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/

bcrypt documentation: https://pypi.org/project/bcrypt/

Flask security guide: https://flask.palletsprojects.com/en/3.0.x/security/

*— End of Report —*